Reverse Proxy Configuration

the settings that matter in production

Watch the 4-min video

In one line: a reverse proxy routes requests to named upstreams, and in production you must set buffering, connect/read/client timeouts that are shorter than the caller's, bounded retries for idempotent requests only, connection and rate limits, body size limits, compression and request logging, because the defaults let one slow dependency take everything down.

1 Defaults are not a configuration

You put NGINX in front of your API, keep the default config, and ship it. Then a slow client ties up your workers, a dead backend makes every request hang for a minute (NGINX's default `proxy_read_timeout` is 60 s), and one retry storm takes everything down. A reverse proxy is only as good as its configuration.

A **reverse proxy** sits in front of your servers and handles requests on their behalf (Phase 2, lesson 27). Behind it are **upstreams** (NGINX) or **clusters** (Envoy): named groups of backend servers. The proxy matches each request by host and path to an upstream, picks a healthy server, and forwards it.

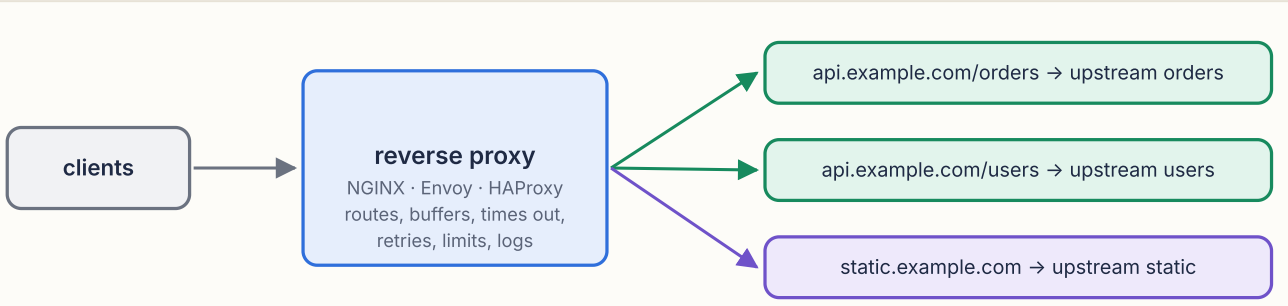


Figure 1. One proxy, many routes: each host and path maps to a named upstream group of servers.

2 Routing and buffering

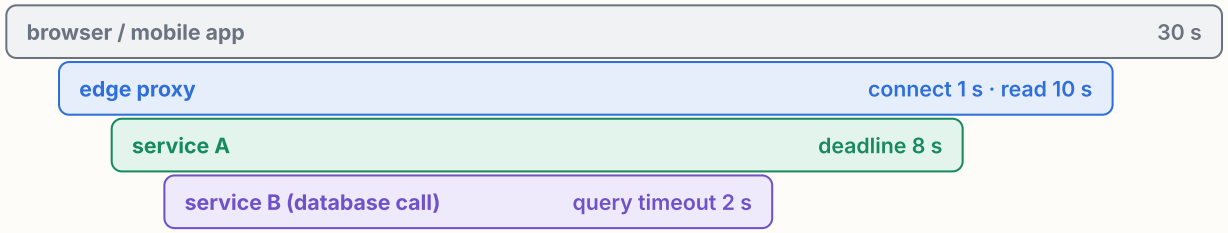
Buffering protects your application workers from slow networks. With **request buffering**, the proxy reads a slow client's whole upload first, then sends it to the backend in one go. With **response buffering**, the backend writes its response to the proxy quickly and moves on, while the proxy trickles it to a phone on a weak network. Either way, a backend worker isn't held hostage for 20 seconds.

Exception: **streaming**. For server-sent events or LLM tokens, turn response buffering off (`proxy_buffering off`, or `X-Accel-Buffering: no`), or users see nothing until the end.

3 Timeouts

- **Connect timeout**: 1 to 2 seconds. A dead backend should fail fast, not hang.
- **Read (response) timeout**: fits your slowest *normal* request, such as 5 or 10 seconds, not a default of a minute.
- **Client timeouts**: drop idle or extremely slow clients (header and body read timeouts, idle keep-alive timeout).

The rule: **every timeout is shorter than its caller's**. Then the inner call gives up first, and there's still time to return a clean error or a fallback.



each hop's timeout is shorter than its caller's, so the inner call gives up first and an error can still be returned

Figure 2. Timeouts nest: each layer gives up before the layer that called it.

4 Retries, with a budget

Retries hide brief failures, but they multiply load. Retry only **idempotent** requests (GETs, or writes with an idempotency key), only on connection errors or specific codes such as 503, cap attempts (1 or 2 retries), give each try its own **per-try timeout**, and use a **retry budget**: retries may never exceed, say, 20% of traffic. Unlimited retries at every layer turn a small outage into a **retry storm**.

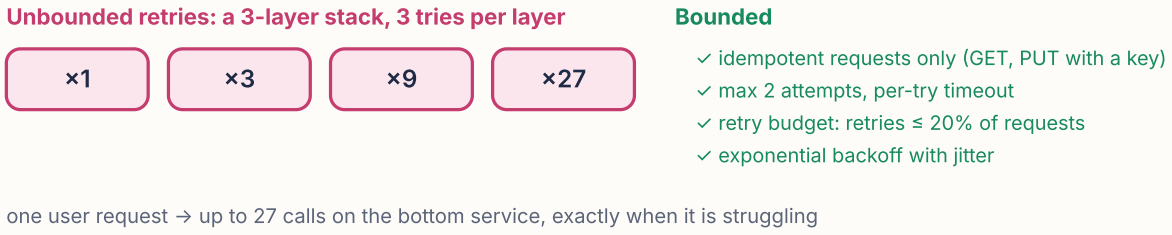


Figure 3. Retries multiply across layers; bounded retries with a budget keep a hiccup from becoming an outage.

5 Protect the backends

- **Connection limits** per upstream, and **keep-alive pools** to upstreams so you don't pay a new TCP and TLS handshake per request.
- **Rate limits** per client IP or API key.
- **Body size limit** (`client_max_body_size`), so one 2 GB upload can't fill your disk or memory.
- **Compression** (gzip or brotli) for text responses such as JSON and HTML.
- **Logging**: a request ID on every request (and passed to the backend), plus upstream connect and response times, so slow requests can be traced.

WORKED EXAMPLE: POLICY IN AN ENVOY SIDECAR

An Envoy sidecar runs next to each service. The calling code makes a plain HTTP call to `http://payments`. The sidecar applies the policy from config: a **2-second per-try timeout**, up to **2 retries** on connection failures and 503s (with backoff), and a **circuit breaker** that caps pending requests and connections to payments. The code stays simple, the policy is the same for every language, and it can be changed without a redeploy.

COMMON MISTAKES

- Inner timeouts longer than outer ones, so the caller gives up while the work continues uselessly.
- Retrying POSTs that create orders or charge cards, causing duplicates.
- Buffering on for streaming endpoints, so tokens arrive all at once at the end.
- No request IDs, so nobody can match a slow request across the proxy and the services.

6 Key terms

Upstream / cluster: a named group of backend servers

Request buffering: read the whole request before forwarding

Response buffering: take the response fast, deliver it slowly

Connect timeout: how long to wait to open a connection

Read timeout: how long to wait for the response

Per-try timeout: a limit on each single attempt

Retry budget: retries capped as a share of traffic

Circuit breaker: stop calling a failing backend for a while

7 Interview questions

Q1. What should you configure on a reverse proxy in production? BASIC

Routing to upstreams, request and response buffering, connect, read and client timeouts, bounded retries for idempotent requests only, connection limits, rate limits, body size limits, compression and request logging.

Q2. Why not leave the defaults? BASIC

Default timeouts are too long and retries can multiply load, so a slow dependency ties up everything and can take the whole system down.

Q3. A backend died and every request hung for 60 seconds. What do you change? SCENARIO

Set a short connect timeout (1 to 2 s) and a realistic read timeout, add health checks or outlier ejection to stop sending to the dead backend, and return a fast error.

Q4. Users of your AI chat see the whole answer appear at once after 20 seconds. Why? SCENARIO

The proxy buffers the response. Turn off response buffering for that streaming route (and make sure compression or other middleware doesn't buffer either).

Q5. Why must inner timeouts be shorter than outer ones? TRICKY

If the inner call waits longer than its caller, the caller has already given up and the inner work is wasted; worse, the caller may retry, doubling the load. Shorter inner timeouts let the service fail fast and still answer.

Q6. How do retries turn a small outage into a big one? TRICKY

Each layer retries, so attempts multiply ($3 \times 3 \times 3 = 27$) against the struggling service, exactly when it has the least capacity. Retry budgets, per-try timeouts, backoff with jitter and retrying at only one layer prevent this.

Next: SD-G-010 · Traffic Shaping and Load Shedding backendroadmap.com